MLA02 – FUNDAMENTALS OF MACHINE LEARNING
QUESTION 6

Naive Bayes Classification

Introduction

Naive Bayes uses Bayes' theorem and assumes conditional independence between features given the class. GaussianNB is suitable for numeric input features.

Implementation library / technique: GaussianNB.

General workflow: prepare the data, separate features and target labels, train the model, classify a new sample, and display the result.

Problem 1: Bank Loan Prediction

Aim

To implement Naive Bayes Classification for bank loan prediction and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Income, CreditScore, LoanAmount and their class labels to train a model, then classify a new sample.

Algorithm

1. Import the required Python libraries.
2. Store the training samples in a feature matrix X and labels in y.
3. Create the classifier and fit it using X and y.
4. Prepare the new sample and apply the same preprocessing, if required.
5. Predict and display the class of the new sample.

Input (Training Data)

Income	CreditScore	LoanAmount	Target
25000	580	8000	0
30000	620	10000	0
45000	700	15000	1
60000	760	18000	1
28000	590	9000	0
52000	730	16000	1
35000	640	11000	0
70000	790	20000	1
40000	680	12000	1
26000	570	7000	0

Program

```
from sklearn.naive_bayes import GaussianNB
import numpy as np

attributes = ['Income', 'CreditScore', 'LoanAmount']
training_data = [[25000, 580, 8000, 0], [30000, 620, 10000, 0], [45000, 700, 15000, 1], [60000, 760, 18000, 1], [28000, 590, 9000, 0], [52000, 730, 16000, 1], [35000, 640, 11000, 0], [70000, 790, 20000, 1], [40000, 680, 12000, 1], [26000, 570, 7000, 0]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
model = GaussianNB()
model.fit(X, y)
new_sample = np.array([55000, 740, 17000], dtype=float)
prediction = model.predict(new_sample)
print("Predicted class:", prediction[0])
print("Training accuracy: {:.2f}".format(model.score(X, y)))
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Naive Bayes Classification was implemented for bank loan prediction and the new sample was classified using the trained model.

Problem 2: Iris Flower Classification

Aim

To implement Naive Bayes Classification for iris flower classification and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes SepalLength, SepalWidth, PetalLength, PetalWidth and their class labels to train a model, then classify a new sample.

Algorithm

6. Import the required Python libraries.
7. Store the training samples in a feature matrix X and labels in y.
8. Create the classifier and fit it using X and y.
9. Prepare the new sample and apply the same preprocessing, if required.
10. Predict and display the class of the new sample.

Input (Training Data)

SepalLength	SepalWidth	PetalLength	PetalWidth	Target
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
5.0	3.4	1.5	0.2	0
6.0	2.7	4.5	1.5	1
5.9	3.0	4.2	1.5	1
6.1	2.8	4.7	1.2	1
6.5	3.0	5.5	1.8	2
6.7	3.1	5.6	2.4	2
6.3	2.9	5.4	1.9	2

Program

```
from sklearn.naive_bayes import GaussianNB
import numpy as np

attributes = ['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth']
training_data = [[5.1, 3.5, 1.4, 0.2, 0], [4.9, 3.0, 1.4, 0.2, 0], [5.0, 3.4, 1.5, 0.2, 0], [6.0, 2.7, 4.5, 1.5, 1], [5.9, 3.0, 4.2, 1.5, 1], [6.1, 2.8, 4.7, 1.2, 1], [6.5, 3.0, 5.5, 1.8, 2], [6.7, 3.1, 5.6, 2.4, 2], [6.3, 2.9, 5.4, 1.9, 2]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
model = GaussianNB()
model.fit(X, y)
new_sample = np.array([5.2, 3.4, 1.5, 0.2], dtype=float)
prediction = model.predict(new_sample)
print("Predicted class:", prediction[0])
print("Training accuracy: {:.2f}".format(model.score(X, y)))
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Naive Bayes Classification was implemented for iris flower classification and the new sample was classified using the trained model.

Problem 3: Email Spam Detection

Aim

To implement Naive Bayes Classification for email spam detection and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Links, SuspiciousWords, MessageLength and their class labels to train a model, then classify a new sample.

Algorithm

11. Import the required Python libraries.
12. Store the training samples in a feature matrix X and labels in y.
13. Create the classifier and fit it using X and y.
14. Prepare the new sample and apply the same preprocessing, if required.
15. Predict and display the class of the new sample.

Input (Training Data)

Links	SuspiciousWords	MessageLength	Target
0	0	40	0
1	0	80	0
5	4	120	1
6	5	150	1
0	1	60	0
4	3	100	1
1	0	55	0
7	6	180	1
2	1	90	0
5	5	130	1

Program

```
from sklearn.naive_bayes import GaussianNB
import numpy as np

attributes = ['Links', 'SuspiciousWords', 'MessageLength']
training_data = [[0, 0, 40, 0], [1, 0, 80, 0], [5, 4, 120, 1], [6, 5, 150, 1], [0, 1, 60, 0], [4, 3, 100, 1], [1, 0, 55, 0], [7, 6, 180, 1], [2, 1, 90, 0], [5, 5, 130, 1]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
model = GaussianNB()
model.fit(X, y)
new_sample = np.array([4, 4, 110], dtype=float)
prediction = model.predict(new_sample)
print("Predicted class:", prediction[0])
print("Training accuracy: {:.2f}".format(model.score(X, y)))
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Naive Bayes Classification was implemented for email spam detection and the new sample was classified using the trained model.

Problem 4: Weather Activity Prediction

Aim

To implement Naive Bayes Classification for weather activity prediction and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Temperature, Humidity, WindSpeed and their class labels to train a model, then classify a new sample.

Algorithm

16. Import the required Python libraries.
17. Store the training samples in a feature matrix X and labels in y.
18. Create the classifier and fit it using X and y.
19. Prepare the new sample and apply the same preprocessing, if required.
20. Predict and display the class of the new sample.

Input (Training Data)

Temperature	Humidity	WindSpeed	Target
30	40	5	1
32	35	4	1
25	70	12	0
22	80	15	0
28	45	6	1
24	75	10	0
31	50	7	1
20	85	18	0
29	55	8	1
23	78	14	0

Program

```
from sklearn.naive_bayes import GaussianNB
import numpy as np

attributes = ['Temperature', 'Humidity', 'WindSpeed']
training_data = [[30, 40, 5, 1], [32, 35, 4, 1], [25, 70, 12, 0], [22, 80, 15, 0], [28, 45, 6, 1], [24, 75, 10, 0], [31, 50, 7, 1], [20, 85, 18, 0], [29, 55, 8, 1], [23, 78, 14, 0]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
model = GaussianNB()
model.fit(X, y)
new_sample = np.array([27, 50, 6], dtype=float)
prediction = model.predict(new_sample)
print("Predicted class:", prediction[0])
print("Training accuracy: {:.2f}".format(model.score(X, y)))
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Naive Bayes Classification was implemented for weather activity prediction and the new sample was classified using the trained model.

Problem 5: Medical Risk Screening Demo

Aim

To implement Naive Bayes Classification for medical risk screening demo and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Age, Glucose, BMI and their class labels to train a model, then classify a new sample.

Algorithm

21. Import the required Python libraries.
22. Store the training samples in a feature matrix X and labels in y.
23. Create the classifier and fit it using X and y.
24. Prepare the new sample and apply the same preprocessing, if required.
25. Predict and display the class of the new sample.

Input (Training Data)

Age	Glucose	BMI	Target
22	85	21	0
30	90	23	0
45	150	31	1
55	170	34	1
28	95	24	0
60	180	36	1
35	105	26	0
50	160	32	1
40	110	27	0
65	190	38	1

Program

```
from sklearn.naive_bayes import GaussianNB
import numpy as np

attributes = ['Age', 'Glucose', 'BMI']
training_data = [[22, 85, 21, 0], [30, 90, 23, 0], [45, 150, 31, 1], [55, 170, 34, 1], [28, 95, 24, 0], [60, 180, 36, 1], [35, 105, 26, 0], [50, 160, 32, 1], [40, 110, 27, 0], [65, 190, 38, 1]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
model = GaussianNB()
model.fit(X, y)
new_sample = np.array([48, 155, 30], dtype=float)
prediction = model.predict(new_sample)
print("Predicted class:", prediction[0])
print("Training accuracy: {:.2f}".format(model.score(X, y)))
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Naive Bayes Classification was implemented for medical risk screening demo and the new sample was classified using the trained model.

Conclusion

Five different application problems were demonstrated using Naive Bayes Classification. The results show how a trained machine-learning model can use example data to classify a new observation. Performance depends on the quality and size of the training data.

Requirements

Python 3, scikit-learn, NumPy, and a Python IDE or Jupyter Notebook.